Part 3 – Integration Testing Documentation

Project Context

This section presents the **integration testing activities** carried out for the **Smart Library System**. Integration testing is used to verify that **multiple components work correctly together**, after their individual correctness has been confirmed through unit testing.

Purpose of Integration Testing

The main goals of integration testing in this project are to:

- Verify the correct interaction between integrated components
- Ensure data flows correctly across different system layers
- Detect interface and interaction defects between modules
- Validate that combined components behave as expected in real scenarios

Integration Testing Approach

Integration testing for the Smart Library System was performed using a **bottom-up strategy**, which involves:

- Starting from low-level components and gradually integrating higher-level modules
- Testing real interactions between components without mocking previously tested units
- Ensuring each integration level is validated before moving to the next one
- Building confidence incrementally as the system grows in complexity

The integration levels were structured as follows, where each higher level assumes that all lower levels have already been successfully tested :

- **Level A:** Memory storage integration
- **Level B:** Policy integration
- **Level C:** CirculationService integration
- **Level D:** LibraryFacade integration

Test Organization and Responsibilities

To ensure clarity and consistency:

- Integration tests follow a clear **layered structure** aligned with the system architecture
- Each team member was assigned responsibility for specific integration levels or classes
- The assignment was based on the **number and complexity of test cases**, ensuring a balanced workload
- Each team member implemented the integration tests for their assigned components and completed the corresponding documentation tables

Integration Testing Documentation Tables

Class: A_MemoryStoragesIntegrationTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
saveMember_thenFind_returnsSameMember()	Verifies that a saved member can be retrieved correctly from memory storage	Member object returned with correct ID, status, and membership type	Member retrieved successfully
saveItem_thenFind_returnsSameItem()	Ensures library items are saved and fetched correctly	Item returned with correct code and availability	Item retrieved successfully
updateAvailability_changesItemAvailability()	Confirms item availability updates correctly	Item availability becomes false	Availability updated successfully
saveBorrowRecord_thenFindActiveBorrow_returnsRecord()	Checks active borrow retrieval after saving	Active borrow record returned	Record retrieved successfully
findActiveBorrow_whenReturned_returnsNull()	Ensures returned borrows are not considered active	Null returned	Null returned correctly
countActiveBorrows_countsOnlyNotReturned()	Verifies counting of active (non-returned) borrows	Count equals number of active borrows	Correct count returned
hasOverdueBorrows_whenAnyOverdueAndNotReturned_returnsTrue()	Detects overdue borrow records	True returned	Overdue detected
findAll_returnsAllSavedRecords()	Ensures all borrow records are returned	List size equals number saved	All records returned

Class: B_PolicyIntegrationTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
borrowPolicy_withRealMember_rejectsInactive()	Prevents inactive members from borrowing	Borrow denied	Borrow denied
borrowPolicy_withRealMember_rejectsAtCapacity()	Prevents borrowing when limit reached	Borrow denied	Borrow denied
borrowPolicy_withRealMember_rejectsWhenOverdue()	Prevents borrowing when overdue exists	Borrow denied	Borrow denied
borrowPolicy_withRealMember_allowsWhenAllGood()	Allows borrowing when all rules satisfied	Borrow allowed	Borrow allowed
penaltyPolicy_calculatesCorrectFine_for1DayOverdue()	Calculates fine for 1 overdue day	$1.00 fine	Correct fine
penaltyPolicy_calculatesCorrectFine_for5DaysOverdue()	Calculates fine within grace period	$5.00 fine	Correct fine
penaltyPolicy_calculatesCorrectFine_for6DaysOverdue()	Applies higher rate after grace period	$12.00 fine	Correct fine
penaltyPolicy_capsAt100_for50PlusDaysOverdue()	Ensures fine does not exceed cap	$100.00 max	Cap applied correctly

Class: C_CirculationServiceIntegrationTest
Responsible Members: Viola Zeneli & Sidrit Rrustemi

Test Case	Test Description & Reason	Expected Output	Actual Result
Viola Zeneli			
requestBorrow_happyPath_savesRecord_andMarksItemUnavailable()	Verifies that a valid borrow request correctly saves a borrow record and marks the item as unavailable.	Borrow record saved and item availability set to false	Borrow record saved and item marked unavailable
requestBorrow_whenMemberNotFound_throwsIllegalStateException()	Ensures borrowing fails when the member does not exist in storage.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_whenItemNotFound_throwsIllegalStateException()	Verifies borrowing fails when the requested item does not exist.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_whenItemNotAvailable_throwsIllegalStateException()	Confirms borrowing is blocked when the item is already unavailable.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_whenBorrowPolicyRejects_dueToInactiveMember_throwsIllegalStateException()	Ensures inactive members are prevented from borrowing items.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_whenBorrowPolicyRejects_dueToLimitReached_throwsIllegalStateException()	Verifies borrowing is denied when the member has reached the maximum loan limit.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_whenBorrowPolicyRejects_dueToOverdue_throwsIllegalStateException()	Confirms borrowing is denied when the member has overdue items.	IllegalStateException thrown	IllegalStateException thrown

returnItem_happyPath_marksReturned_andMarksItemAvailable()	Verifies that returning an item marks the borrow as returned and restores item availability.	Record marked returned and item available	Record returned and item available
returnItem_whenOverdue_sendsAlert()	Ensures an alert is sent when an overdue item is returned.	Alert sent to member	Alert sent
returnItem_whenNotOverdue_doesNotSendAlert()	Confirms no alert is sent for items returned on time.	No alert sent	No alert sent
canBorrowMore_whenUnderLimit_returnsTrue()	Verifies the service allows borrowing when the member is under the loan limit.	true	true
canBorrowMore_whenAtLimit_returnsFalse()	Ensures borrowing is not allowed when the member has reached the loan limit.	false	false
multipleMembersCanBorrowDifferentItems_simultaneously()	Verifies that multiple members can borrow different items at the same time.	Both borrow operations succeed	Both borrows succeed
sameMember_cannotBorrowSameItemTwice()	Confirms a member cannot borrow the same item more than once simultaneously.	IllegalStateException thrown	IllegalStateException thrown
borrowReturnBorrowAgain_sameItem_worksCorrectly()	Ensures an item can be borrowed again after it has been returned.	Second borrow succeeds	Borrowing and returning the item functions correctly.
borrowMultipleItems_respectsCapacity()	Verifies borrowing multiple items respects membership capacity limits.	IllegalStateException thrown when limit exceeded	IllegalStateException thrown

Sidrit Rrustemi			
calculateFine_usesRealPenaltyPolicy_withOverdueRecord()	Verifies that fine calculation uses the real penalty policy for overdue items	Fine of $3.0 for 3 days overdue	Fine calculated as $3.0
calculateFine_returns0_whenNotOverdue()	Verifies that no fine is charged when item is not overdue	Fine of $0.0	Fine returned as $0.0
getRemainingBorrowCapacity_decreasesAfterBorrow_increasesAfterReturn()	Verifies that borrow capacity decreases after borrowing and increases after returning	Capacity: 3 → 2 → 3	Capacity changes correctly through lifecycle: 3 → 2 → 3
validateMemberStatus_reflectsRealStorageState()	Verifies that member status validation reflects actual storage state including overdue items	Status changes from ACTIVE_GOOD_STANDING to OVERDUE_ITEMS	Status reflects real-time storage state
canRenewBorrow_checksRealOverdueStatus()	Verifies that renewal eligibility checks actual overdue status from storage	Returns true when no overdue, false when overdue items exist	Renewal eligibility correctly based on overdue status: true→false as expected
getFineCategory_correctForDifferentOverdueLevels()	Verifies that fine categories are correctly assigned based on overdue duration	NO_FINE_ON_TIME, LOW_FINE, MEDIUM_FINE categories	Categories assigned correctly per overdue level
getDaysUntilDue_countdownToZero_thenNegative()	Verifies that days until due counts down and becomes negative when overdue	Positive for future dates, negative for past dates	Days calculated correctly including negative values: Days: +3, -2
calculateLateFeeWithDiscount_appliesCorrectly()	Verifies that discount percentage is correctly applied to calculated fines	$20.0 fine with 50% discount = $10.0	Discount applied correctly to fine: $20.0→$10.0

getMembershipEligibility_basedOnBorrowHistory()	Verifies that membership eligibility is determined by borrow count and overdue status	Different eligibility levels based on usage patterns	Eligibility categories assigned correctly
assessItemReturnStatus_progressesThroughCategories()	Verifies that return status categories progress correctly based on days overdue	ON_TIME → SLIGHTLY_LATE → MODERATELY_LATE → VERY_LATE → SEVERELY_OVERDUE	Status categories progress correctly
canBorrowItemType_respectsValueLimits()	Verifies that item value limits are enforced based on membership type	STUDENT max $100, GUEST max $50 enforced	Value limits respected per membership type
shouldSendReminder_triggersAtRightTime()	Verifies that reminders are triggered at appropriate times before/after due date	Reminders sent at threshold days and for overdue items	Reminder triggers work correctly
memberBecomesInactive_cannotBorrowMore()	Verifies that inactive members cannot borrow even if previously active	Exception thrown when inactive member attempts to borrow	Inactive status prevents borrowing: IllegalStateException thrown
itemBecomesUnavailable_afterFirstBorrow()	Verifies that borrowed items cannot be borrowed by other members	Exception thrown when attempting to borrow unavailable item	Unavailable items cannot be borrowed: IllegalStateException thrown
isEligibleToBorrow_checksAllConditions_withRealData()	Verifies that eligibility checks consider all conditions including capacity limits	True when eligible, false when at capacity	All eligibility conditions checked correctly: true→false at capacity
getTotalOverdueDays_sumsAcrossMultipleBorrows()	Verifies that overdue days are correctly summed across multiple active borrows	Total of 8 days (3 + 5, ignoring non-overdue)	Overdue days summed correctly: Total = 8 days

Class: D_LibraryFacadeIntegrationTest
Responsible Member: Arsildo Veliu

Test Case	Test Description & Reason	Expected Output	Actual Result
borrowFlow_delegatesToCirculationService_andBorrowIsProcessed()	Verifies borrow flow updates item and borrow record	Item unavailable, borrow created	Borrow successful
returnFlow_delegatesToCirculationService_andReturnIsProcessed()	Verifies return updates item and borrow record	Item available, borrow cleared	Return successful
borrowThenReturn_fullFlow_updatesAvailability_andBorrowRecordStatus()	Ensures full borrow-return lifecycle works	Record marked returned	Flow completed
borrowFlow_memberNotFound_throwsRuntimeException()	Prevents borrowing with invalid member	Exception thrown	Exception thrown
borrowFlow_itemNotFound_throwsRuntimeException()	Prevents borrowing invalid item	Exception thrown	Exception thrown
borrowFlow_itemNotAvailable_throwsRuntimeException()	Prevents borrowing unavailable item	Exception thrown	Exception thrown
borrowFlow_inactiveMember_throwsRuntimeException()	Prevents inactive members from borrowing	Exception thrown	Exception thrown

borrowFlow_borrowLimitReached_throwsRuntimeException()	Enforces borrowing limits	Exception thrown	Exception thrown
borrowFlow_memberHasOverdue_throwsRuntimeException()	Blocks borrowing with overdue items	Exception thrown	Exception thrown
returnFlow_notOverdue_doesNotSendAlert()	Ensures no alert for on-time return	No alert sent	No alert sent
returnFlow_overdue_sendsAlertToCorrectMemberWithMessage()	Sends alert for overdue return	Alert sent with message	Alert sent
returnFlow_noActiveBorrow_throwsRuntimeException()	Prevents returning non-borrowed item	Exception thrown	Exception thrown